3. The Programming Interface

From a programmer's point of view, the user is a peripheral that types when you issue a read request. —*Peter Williams*

The previous chapter concerned the mechanisms needed in the operating system kernel to implement the process abstraction. A process is an instance of a program — the kernel provides an efficient sandbox for executing untrusted code at user-level, running user code directly on the processor.

This chapter concerns how we choose to use the process abstraction: what functionality does the operating system provide applications, and what should go where — what functionality should be put in the operating system kernel, what should be put into user-level libraries, and how should the operating system itself be organized?

There are as many answers to this as there are operating systems. Describing the full programming interface and internal organization for even a single operating system would take an entire book. Instead, in this chapter we explore a subset of the programming interface for UNIX, the foundation of Linux, MacOS, iOS, and Android. We also touch on how the same issues are addressed in Windows.

First, we need to answer "what" — what functions do we need an operating system to provide applications?

- **Process management.** Can a program create an instance of another program? Wait for it to complete? Stop or resume another running program? Send it an asynchronous event?
- **Input/output.** How do processes communicate with devices attached to the computer and through them to the physical world? Can processes communicate with each other?
- **Thread management.** Can we create multiple activities or threads that share memory or other resources within a process? Can we stop and start threads? How do we synchronize their use of shared data structures?
- **Memory management.** Can a process ask for more (or less) memory space? Can it share the same physical memory region with other processes?
- **File systems and storage.** How does a process store the user's data persistently so that it can survive machine crashes and disk failures? How does the user name and organize their data?

- **Networking and distributed systems.** How do processes communicate with processes on other computers? How do processes on different computers coordinate their actions despite machine crashes and network problems?
- **Graphics and window management.** How does a process control pixels on its portion of the screen? How does a process make use of graphics accelerators?
- **Authentication and security.** What permissions does a user or a program have, and how are these permissions kept up to date? On what basis do we know the user (or program) is who they say they are?

In this chapter, we focus on just the first two of these topics: process management and input/output. We will cover thread management, memory management, and file systems in detail in later chapters in this book.

Remarkably, we can describe a functional interface for process management and input/output with just a dozen system calls, and the rest of the system call interface with another dozen. Even more remarkably, these calls are nearly unchanged from the original UNIX design. Despite being first designed and implemented in the early 1970's, most of these calls are still in wide use in systems today!

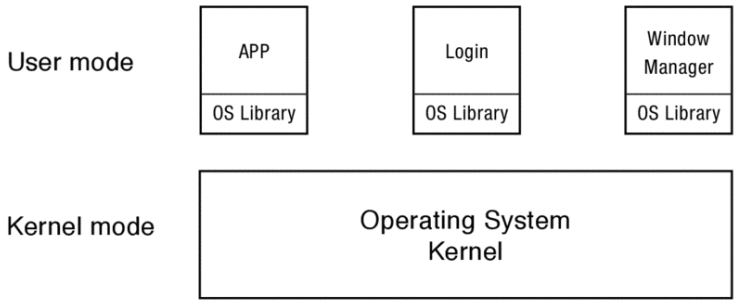


Figure 3.1: Operating system functionality can be implemented in user-level programs, in user-level libraries, in the kernel itself, or in a user-level server invoked by the kernel.

Second, we need to answer "where" — for any bit of functionality the operating system provides to user programs, we have several options for where it lives, illustrated in Figure 3.1:

- We can put the functionality in a user-level program. In both Windows and UNIX, for example, there is a user program for managing a user's login and another for managing a user's processes.
- We can put the functionality in a user-level library linked in with each application. In Windows and MacOS, user interface widgets are part of user-level libraries, included in those applications that need them.

- We can put the functionality in the operating system kernel, accessed through a system call. In Windows and UNIX, low-level process management, the file system and the network stack are all implemented in the kernel.

- We can access the function through a system call, but implement the function in a standalone server process invoked by the kernel. In many systems, the window manager is implemented as a separate server process.

How do we make this choice? It is important to realize that the choice can be (mostly) transparent to both the user and the application programmer. The user wants a system that works; the programmer wants a clean, convenient interface that does the job. As long as the operating system provides that interface, where each function is implemented is up to the operating system, based on a tradeoff between flexibility, reliability, performance, and safety.

- **Flexibility.** It is much easier to change operating system code that lives outside of the kernel, without breaking applications using the old interface. If we create a new version of a library, we can just link that library in with new applications, and over time convert old applications to use the new interface. However, if we need to change the system call interface, we must either simultaneously change both the kernel and all applications, or we must continue to support both the old and the new versions until all old applications have been converted. Many applications are written by third party developers, outside of the control of the operating system vendor. Thus, changing the system call interface is a huge step, often requiring coordination across many companies.

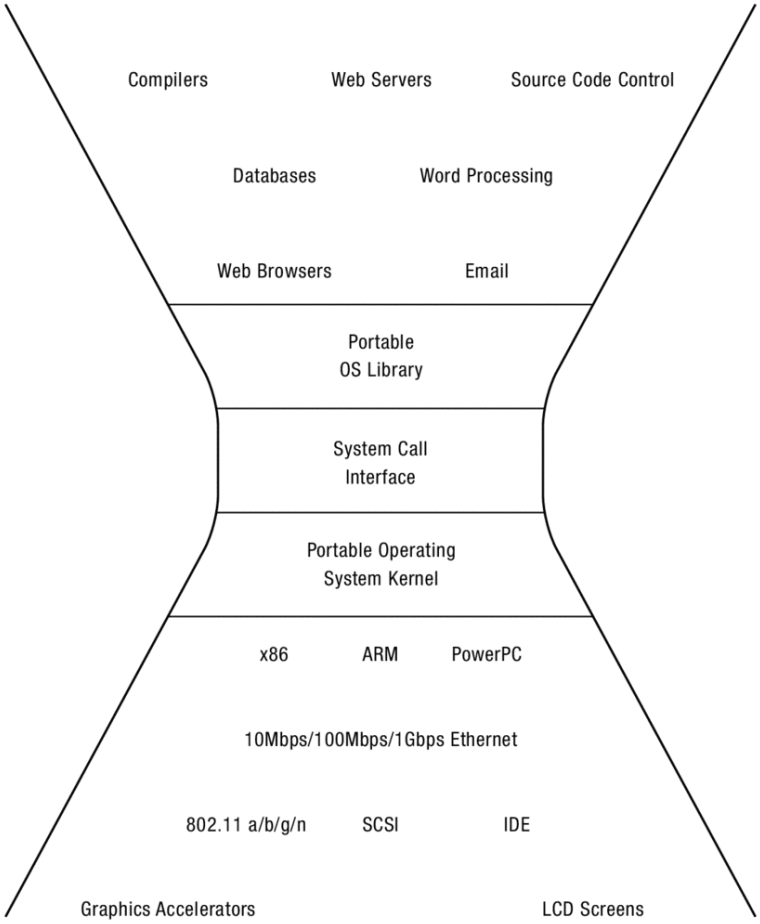


Figure 3.2: The kernel system call interface can be seen as a “thin waist,” enabling independent evolution of applications and hardware.

One of the key ideas in UNIX, responsible for much of its success, was to design its system call interface to be simple and powerful, so that almost all of the innovation in the system could happen in user code without changing the interface to the operating system. The UNIX system call interface is also highly portable — the operating system can be ported to new hardware without needing to rewrite application code. As shown in Figure 3.2, the kernel can be seen as a “thin waist,” enabling innovation at the application-level, and in the hardware, without requiring simultaneous changes in the other parts of the system.

The Internet and the “thin waist”

The Internet is another example of the benefit of designing interfaces to be simple and portable. The Internet defines a packet-level protocol that can run on top of virtually any type of network hardware and can support almost any type of network application. Creating the World Wide Web required no

changes to the Internet packet delivery mechanism; likewise, the introduction of wireless networks required changes in hardware devices and in the operating system, but no changes in network applications. Although the Internet's "thin waist" can sometimes lead to inefficiencies, the upside is to foster innovation in both applications and hardware by decoupling changes in one from changes in the other.

- **Safety.** However, resource management and protection are the responsibility of the operating system kernel. As Chapter 2 explained, protection checks cannot be implemented in a user-level library because application code can skip any checks made by the library.

- **Reliability.** Improved reliability is another reason to keep the operating system kernel minimal. Kernel code needs the power to set up hardware devices, such as the disk, and to control protection boundaries between applications. However, kernel modules are typically not protected from one another, and so a bug in kernel code (whether sensitive or not) may corrupt user or kernel data. This has led some systems to use a philosophy of "what can be at user level, should be." An extreme version of approach is to isolate privileged, but less critical, parts of the operating system such as the file system or the window system, from the rest of the kernel. This is called a *microkernel* design. In a microkernel, the kernel itself is kept small, and instead most of the functionality of a traditional operating system kernel is put into a set of user-level processes, or servers, accessed from user applications via interprocess communication.

- **Performance.** Finally, transferring control into the kernel is more expensive than a procedure call to a library, and transferring control to a user-level file system server via the kernel is still even more costly. Hardware designers have attempted to reduce the cost of these boundary crossings, but their performance remains a problem. Microsoft Windows NT, a precursor to Windows 7, was initially designed as a microkernel, but over time, much of its functionality has been migrated back into the kernel for performance reasons.

Application-level sandboxing and operating system functionality

Applications that support executing third-party code or scripts in a restricted sandbox must address many of these same questions, with the sandbox playing the role of the operating system kernel. In terms of functionality: Can the scripting code start a new instance of itself? Can it do input/output? Can it perform work in the background? Can it store data persistently, and if it can, how does it name that data? Can it communicate data over the network? How does it authenticate actions?

For example, in web browsers, HTML5 not only allows scripts to draw on the screen, communicate with servers, and save and read cookies, it also has recently added programming interfaces for offline storage and cross-document communication. The Flash media player provides scripts with the ability to do asynchronous operations, file storage, network communication, memory management, and authentication.

Just as with system calls, these interfaces must be carefully designed to be bulletproof against malicious use. A decade ago, email viruses became widespread because scripts could be embedded in documents that were executed on opening; the programming interfaces for these scripts would allow them to discover the list of correspondents known to the current email user and to send them email, thereby propagating and

expanding the virus with a single click. The more fully featured the interface, the more convenient it is for developers, and the more likely that some aspect of the interface will be abused by a hacker.

There are no easy answers! We will investigate the question of how to design the system call interface and where to place operating system functionality through case studies of UNIX and other systems.

Chapter roadmap:

- **Process management.** What is the system call interface for process management? (Section 3.1)
- **Input/output.** What is the system call interface for performing I/O and interprocess communication? (Section 3.2)
- **Case study: Implementing a shell.** We will illustrate these interfaces by using them to implement a user-level job control system called a *shell*. (Section 3.3)
- **Case study: Interprocess communication.** How does the communication between a client and server work? (Section 3.4)
- **Operating system structure.** Can we use the process abstraction to simplify the construction of the operating system itself and to make it more secure, more reliable, and more flexible? (Section 3.5)

3.1 Process Management

On a modern computer, when a user clicks on a file or application icon, the application starts up. How does this happen and who is called? Of course, we could implement everything that needs to happen in the kernel — draw the icon for every item in the file system, map mouse positions to the intended icon, catch the mouse click, and start the process. In early batch processing systems, the kernel was in control by necessity. Users submitted jobs, and the operating system took it from there, instantiating the process when it was time to run the job.

A different approach is to allow user programs to create and manage their own processes. This has fostered a blizzard of innovation. Today, programs that create and manage processes include window managers, web servers, web browsers, shell command line interpreters, source code control systems, databases, compilers, and document preparation systems. We could go on, but you get the idea. If creating a process is something a process can do, then anyone can build a new version of any of these applications, without recompiling the kernel or forcing anyone else to use it.

An early motivation for user-level process management was to allow developers to write their own shell command line interpreters. A *shell* is a job control system; both Windows and UNIX have a shell. Many tasks involve a sequence of steps to do something, each of which can be its own program. With a shell, you can write down the sequence of steps, as

a sequence of programs to run to do each step. Thus, you can view it as a very early version of a scripting system.

For example, to compile a C program from multiple source files, you might type:

```
cc -c sourcefile1.c
cc -c sourcefile2.c
ln -o program sourcefile1.o sourcefile2.o
```

If we put those commands into a file, the shell reads the file and executes it, creating, in turn, a process to compile sourcefile1.c, a process to compile sourcefile2, and a process to link them together. Once a shell script is a program, we can create other programs by combining scripts together. In fact, on UNIX, the C compiler is itself a shell program! The compiler first invokes a process to expand header include files, then a separate process to parse the output, another process to generate (text) assembly code, and yet another to convert assembly into executable machine instructions.

There is an app for that

User-level process management is another way of saying “there is an app for that.” Instead of a single program that does everything, we can create specialized programs for each task, and mix and match what we need. The formatting system for this textbook uses over fifty separate programs.

The web is a good example of the power of composing complex applications from more specialized services. A web page does not need to do everything itself: it can mash up the results of many different web pages, and it can invoke process creation on the local server to generate part of the page. The flexibility to create processes was extremely important early on in the development of the web. HTML was initially just a way to describe the formatting for static information, but it included a way to escape to a process, e.g., to do a lookup in a database or to authenticate a user. Over time, HTML has added support for many different features that were first prototyped via execution by a separate process. And of course, HTML can still execute a process for any format not supported by the standard.

3.1.1 Windows Process Management

One approach to process management is to just add a system call to create a process, and other system calls for other process operations. This turns out to be simple in theory and complex in practice. In Windows, there is a routine called, unsurprisingly, CreateProcess, in simplified form below:

```
boolean CreateProcess(char *prog, char *args);
```

We call the process creator the *parent* and the process being created, the *child*.

What steps does CreateProcess take? As we explained in the previous chapter, the kernel needs to:

- Create and initialize the process control block (PCB) in the kernel.
- Create and initialize a new address space.
- Load the program prog into the address space.
- Copy arguments args into memory in the address space.
- Initialize the hardware context to start execution at “start”.
- Inform the scheduler that the new process is ready to run.

Unfortunately, there are quite a few aspects of the process that the parent might like to control, such as: its privileges, where it sends its input and output, what it should store its files, what to use as a scheduling priority, and so forth. We cannot trust the child process itself to set its own privileges and priority, and it would be inconvenient to expect every application to include code for figuring out its context. So the real interface to CreateProcess is quite a bit more complicated in practice, given in Figure 3.3.

```
// Start the child process
if (!CreateProcess(NULL,   // No module name (use command line)
    argv[1],        // Command line
    NULL,           // Process handle not inheritable
    NULL,           // Thread handle not inheritable
    FALSE,          // Set handle inheritance to FALSE
    0,              // No creation flags
    NULL,           // Use parent's environment block
    NULL,           // Use parent's starting directory
    &si,            // Pointer to STARTUPINFO structure
    &pi )           // Pointer to PROCESS_INFORMATION structure
)
```

Figure 3.3: Excerpt from an example of how to use the Windows CreateProcess system call. The first two arguments specify the program and its arguments; the rest concern aspects of the process runtime environment.

3.1.2 UNIX Process Management

UNIX takes a different approach to process management, one that is complex in theory and simple in practice. UNIX splits CreateProcess in two steps, called fork and exec, illustrated in Figure 3.4.

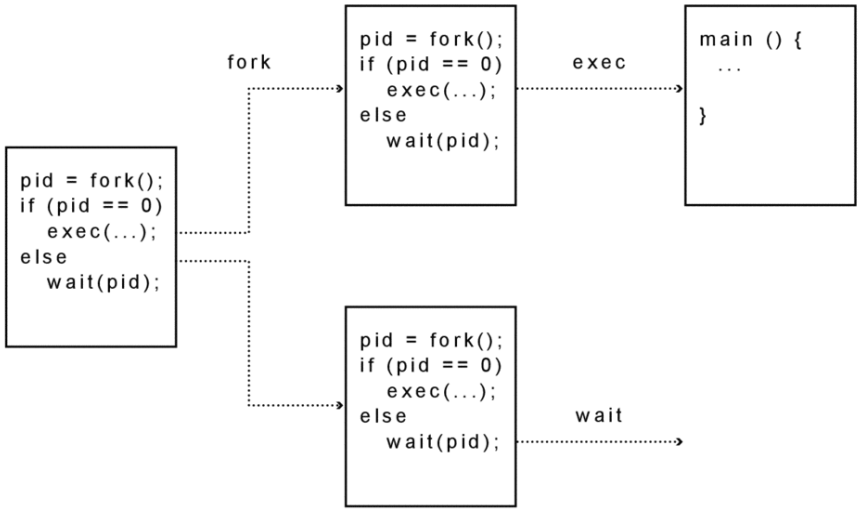


Figure 3.4: The operation of the UNIX fork and exec system calls. UNIX fork makes a copy of the parent process; UNIX exec changes the child process to run the new program.

UNIX fork creates a complete copy of the parent process, with one key exception. (We need some way to distinguish between which copy is the parent and which is the child.) The child process sets up privileges, priorities, and I/O for the program that is about to be started, e.g., by closing some files, opening others, reducing its priority if it is to run in the background, etc. Because the child runs exactly the same code as the parent, it can be trusted to set up the context for the new program correctly.

Once the context is set, the child process calls *UNIX exec*. UNIX exec brings the new executable image into memory and starts it running. It may seem wasteful to make a complete copy of the parent process, just to overwrite that copy when we bring in the new executable image into memory using exec. It turns out that fork and exec can be implemented efficiently, using a set of techniques we will describe in Chapter 8.

With this design, UNIX fork takes no arguments and returns an integer. UNIX exec takes two arguments (the name of the program to run and an array of arguments to pass to the program). This is in place of the ten parameters needed for CreateProcess. In part because of the simplicity of UNIX fork and exec, this interface has remained nearly unchanged since UNIX was designed in the early 70's. (Although the interface has not changed, the word fork is now a bit ambiguous. It is used for creating a new copy of a UNIX process, and in thread systems for creating a new thread. To disambiguate, we will always use the term "UNIX fork" to refer to UNIX's copy process system call.)

UNIX fork

The steps for implementing UNIX fork in the kernel are:

- Create and initialize the process control block (PCB) in the kernel
- Create a new address space
- Initialize the address space with a copy of the entire contents of the address space of the parent
- Inherit the execution context of the parent (e.g., any open files)
- Inform the scheduler that the new process is ready to run

A strange aspect of UNIX fork is that the system call returns *twice*: once to the parent and once to the child. To the parent, UNIX returns the process ID of the child; to the child, it returns zero indicating success. Just as if you made a clone of yourself, you would need some way to tell who was the clone and who was the original, UNIX uses the return value from fork to distinguish the two copies. Some sample code to call fork is given in Figure 3.5.

```
int child_pid = fork();

if (child_pid == 0) {   // I'm the child process.
    printf("I am process #%d\n", getpid());
    return 0;
} else {                // I'm the parent process.
    printf("I am the parent of process #%d\n", child_pid);
    return 0;
}

Possible output:
    I am the parent of process 495
    I am process 495

Another less likely but still possible output:
    I am process 456
    I am the parent of process 456
```

Figure 3.5: Example UNIX code to fork a process, and some possible outputs of running the code. getpid is a system call to get the current process's ID.

If we run the program in Figure 3.5, what happens? If you have access to a UNIX system, you can try it and see for yourself. UNIX fork returns twice, once in the child, with a return value of zero, and once in the parent with a return value of the child's process ID. However, we do not know whether the parent will run next or the child. The parent had been running, and so it is likely that it will reach its print statement first. However, a timer interrupt could intervene between when the parent forks the process and when it reaches the print statement, so that the processor is reassigned to the child. Or we could be running on a multicore system, where both the parent and child are running simultaneously. In either case, the child could print its output before the parent. We will talk in much more depth about the implications of different orderings of concurrent execution in the next chapter.

UNIX fork and the Chrome Web browser

Although UNIX fork is normally paired with a call to exec, in some cases UNIX fork is useful on its own. A particularly interesting example is in Google's Chrome web browser. When the user clicks on a link, Chrome forks a process to fetch and render the web page at the link, in a new tab on the browser. The parent process continues to display the original referring web page, while the child process runs the same browser, but in its own address space and protection boundary. The motivation for this design is to isolate the new link, so that if the web site is infected with a virus, it will not infect the rest of the browser. Closing the infected browser tab will then remove the link and the virus from the system.

Some security researchers take this a step further. They set up their browsers and email systems to create a new *virtual machine* for every new link, running a copy of the browser in each virtual machine; even if the web site has a virus that corrupts the guest operating system running in the virtual machine, the rest of the system will remain unaffected. In this case, closing the virtual machine cleans the system of the virus.

Interestingly, on Windows, Google Chrome does not use CreateProcess to fork new copies of the browser on demand. The difficulty is that if Chrome is updated while Chrome is running, CreateProcess will create a copy of the new version, and that may not interoperate correctly with the old version. Instead, they create a pool of helper processes that wait in the background for new links to render.

UNIX exec and wait

The UNIX system call exec completes the steps needed to start running a new program. The child process typically calls UNIX exec once it has returned from UNIX fork and configured the execution environment for the new process. We will describe more about how this works when we discuss UNIX pipes in the next section.

UNIX exec does the following steps:

- Load the program prog into the current address space.
- Copy arguments args into memory in the address space.
- Initialize the hardware context to start execution at "start."

Note that exec does not create a new process!

On the other side, often the parent process needs to pause until the child process completes, e.g., if the next step depends on the output of the previous step. In the shell example we started the chapter with, we need to wait for the two compilations to finish before it is safe to start the linker.

UNIX has a system call, naturally enough called *wait*, that pauses the parent until the child finishes, crashes, or is terminated. Since the parent could have created many child processes, wait is parameterized with the process ID of the child. With wait, a shell can create a new process to perform some step of its instructions, and then pause for that step to complete before proceeding to the next step. It would be hard to build a usable shell without wait.

However, the call to wait is optional in UNIX. For example, the Chrome browser does not need to wait for its forked clones to finish. Likewise, most UNIX shells have an option to

run operations in the background, signified by appending ‘&’ to the command line. (As with fork, the word wait is now a bit ambiguous. It is used for pausing the current UNIX process to wait for another process to complete; it is also used in thread synchronization, for waiting on a condition variable. To disambiguate, we will always use the term “UNIX wait” to refer to UNIX’s wait system call. Oddly, waiting for a thread to complete is called “thread join”, even though it is most analogous to UNIX wait. Windows is simpler, with a single function called “WaitForSingleObject” that can wait for process completion, thread completion, or on a condition variable.)

Kernel handles and garbage collection

As we discussed in the previous chapter, when a UNIX process finishes, it calls the system call exit. Exit can release various resources associated with the process, such as the user stack, heap, and code segments. It must be careful, however, in how it garbage collects the process control block (PCB). Even though the child process has finished, if it deletes the PCB, then the parent process will be left with a dangling pointer if later on it calls UNIX wait. Of course, we don’t know for sure if the parent will ever call wait, so to be safe, the PCB can only be reclaimed when both the parent and the child have finished or crashed.

Generalizing, both Windows and UNIX have various system calls that return a handle to some kernel object; these handles are used in later calls as an ID. The process ID returned by UNIX fork is used in later calls to UNIX wait; we will see below that UNIX open returns a file descriptor that is used in other system calls. It is important to realize that these handles are *not* pointers to kernel data structures; otherwise, an erroneous user program could cause havoc in the kernel by making system calls with fake handles. Rather, they are specific to the process and checked for validity on each use.

Further, in both Windows and UNIX, handles are reference counted. Whenever the kernel returns a handle, it bumps a reference counter, and whenever the process releases a handle (or exits), the reference counter is decremented. UNIX fork sets the process ID reference count to two, one for the parent and one for the child. The underlying data structure, the PCB, is reclaimed only when the reference count goes to zero, that is, when both the parent and child terminate.

Finally, as we outlined in the previous chapter, UNIX provides a facility for one process to send another an instant notification, or upcall. In UNIX, the notification is sent by calling *signal*. Signals are used for terminating an application, suspending it temporarily for debugging, resuming after a suspension, timer expiration, and a host of other reasons. In the default case, where the receiving application did not specify a signal handler, the kernel implements a standard one on its behalf.

3.2 Input/Output

Computer systems have a wide diversity of input and output devices: keyboard, mouse, disk, USB port, Ethernet, WiFi, display, hardware timer, microphone, camera, accelerometer, and GPS, to name a few.

To deal with this diversity, we could specialize the application programming interface for each device, customizing it to the device’s specific characteristics. After all, a disk device is quite different from a network and both are quite different from a keyboard: a disk is addressed in fixed sized chunks, while a network sends and receives a stream of variable sized packets, and the keyboard returns individual characters as keys are pressed. While

the disk only returns data when asked, the network and keyboard provide data unprompted. Early computer systems took the approach of specializing the interface to the device, but it had a significant downside: every time a new type of hardware device is invented, the system call interface has to be upgraded to handle that device.

One of the primary innovations in UNIX was to regularize all device input and output behind a single common interface. In fact, UNIX took this one giant step further: it uses this same interface for reading and writing files and for interprocess communication. This approach was so successful that it is almost universally followed in systems today. We will sketch the interface in this section, and then in the next section, show how to use it to build a shell.

The basic ideas in the UNIX I/O interface are:

- **Uniformity.** All device I/O, file operations, and interprocess communication use the same set of system calls: open, close, read and write.

- **Open before use.** Before an application does I/O, it must first call open on the device, file, or communication channel. This gives the operating system a chance to check access permissions and to set up any internal bookkeeping. Some devices, such as a printer, only allow one application access at a time — the open call can return an error if the device is in use.

  Open returns a handle to be used in later calls to read, write and close to identify the file, device or channel; this handle is somewhat misleadingly called a *file descriptor*, even when it refers to a device or channel so there is no file involved. For convenience, the UNIX shell starts applications with open file descriptors for reading and writing to the terminal.

- **Byte-oriented.** All devices, even those that transfer fixed-size blocks of data, are accessed with byte arrays. Similarly, file and communication channel access is in terms of bytes, even though we store data structures in files and send data structures across channels.

- **Kernel-buffered reads.** Stream data, such as from the network or keyboard, is stored in a kernel buffer and returned to the application on request. This allows the UNIX system call read interface to be the same for devices with streaming reads as those with block reads, such as disks and Flash memory. In both cases, if no data is available to be returned immediately, the read call blocks until it arrives, potentially giving up the processor to some other task with work to do.

- **Kernel-buffered writes.** Likewise, outgoing data is stored in a kernel buffer for transmission when the device becomes available. In the normal case, the system call write copies the data into the kernel buffer and returns immediately. This decouples the application from the device, allowing each to go at its own speed. If the application generates data faster than the device can receive it (as is common when spooling data to a printer), the write system call blocks in the kernel until there is enough room to store the new data in the buffer.

- **Explicit close.** When an application is done with the device or file, it calls close. This signals to the operating system that it can decrement the reference-count on the device, and garbage collect any unused kernel data structures.

Open vs. creat vs. stat

By default, the UNIX open system call returns an error if the application tries to open a file that does not exist; as an option (not shown above), a parameter can tell the kernel to instead create the file if it does not exist. Since UNIX also has system calls for creating a file (creat) and for testing whether a file exists (stat), it might seem as if open could be simplified to always assume that the file already exists.

However, UNIX often runs in a multi-user, multi-application environment, and in that setting the issue of system call design can become more subtle. Suppose instead of the UNIX interface, we had completely separate functions for testing if a file exists, creating a file, and opening the file. Assuming that the user has permission to test, open, or create the file, does this code work?

```
if (!exists(file)) {    // If the file doesn't exist create it.
// Are we guaranteed the file doesn't exist?
    create(file);
}
// Are we guaranteed the file does exist?
open(file);
```

The problem is that on a multi-user system, some other user might have created the file in between the call to test for its existence, and the call to create the file. Thus, call to create must also test the existence of the file. Likewise, some other user might have deleted the file between the call to create and the call to open. So open also needs the ability to test if the file is there, and if not to create the file (if that is the user's intent).

UNIX addresses this with an all-purpose, atomic open: test if the file exists, optionally create it if it does not, and then open it. Because system calls are implemented in the kernel, the operating system can make open (and all other I/O systems calls) non-interruptible with respect to other system calls. If another user tries to delete a file while the kernel is executing an open system call on the same file, the delete will be delayed until the open completes. The open will return a file descriptor that will continue to work until the application closes the file. The delete will remove the file from the file system, but the file system does not actually reclaim its disk blocks until the file is closed.

For interprocess communication, we need a few more concepts:

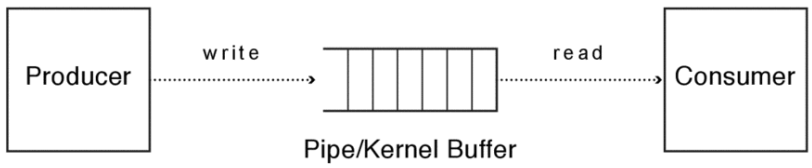


Figure 3.6: A pipe is a temporary kernel buffer connecting a process producing data with a process consuming the data.

- **Pipes.** A *UNIX pipe* is a kernel buffer with two file descriptors, one for writing (to put data into the pipe) and one for reading (to pull data out of the pipe), as illustrated in Figure 3.6. Data is read in exactly the same sequence it is written, but since the data is buffered, the execution of the producer and consumer can be decoupled, reducing waiting in the common case. The pipe terminates when either endpoint closes the pipe or exits.

  The Internet has a similar facility to UNIX pipes called TCP (Transmission Control Protocol). Where UNIX pipes connect processes on the same machine, TCP provides a bi-directional pipe between two processes running on different machines. In TCP, data is written as a sequence of bytes on one machine and read out as the same sequence on the other machine.

- **Replace file descriptor.** By manipulating the file descriptors of the child process, the shell can cause the child to read its input from, or send its output to, a file or a pipe instead of from a keyboard or to the screen. This way, the child process does not need to be aware of who is providing or consuming its I/O. The shell does this redirection using a special system call named dup2(from, to) that replaces the to file descriptor with a copy of the from file descriptor.

- **Wait for multiple reads.** For client-server computing, a server may have a pipe open to multiple client processes. Normally, read will block if there is no data to be read, and it would be inefficient for the server to poll each pipe in turn to check if there is work for it to do. The UNIX system call select(fd[], number) addresses this. Select allows the server to wait for input from any of a set of file descriptors; it returns the file descriptor that has data, but it does not read the data. Windows has an equivalent function, called WaitForMultipleObjects.

Figure 3.7 summarizes the dozen UNIX system calls discussed in this section.

Creating and managing processes	
.	
fork ()	Create a child process as a clone of the current process. The fork call returns to both the parent and child.
exec (prog, args)	Run the application prog in the current process.
exit ()	Tell the kernel the current process is complete, and its data structures should be garbage collected.
wait (processID)	Pause until the child process has exited.
signal (processID, type)	Send an interrupt of a specified type to a process.
I/O operations	
fileDesc open (name)	Open a file, channel, or hardware device, specified by name; returns a file descriptor that can be used by other calls.
pipe (fileDesc[2])	Create a one-directional pipe for communication between two processes. Pipe returns two file descriptors, one for reading and one for writing.

dup2 (fromFileDesc, toFileDesc)	Replace the toFileDesc file descriptor with a copy of fromFileDesc. Used for replacing stdin or stdout or both in a child process before calling exec.
int read (fileDesc, buffer, size)	Read up to size bytes into buffer, from the file, channel, or device. Read returns the number of bytes actually read. For streaming devices this will often be less than size. For example, a read from the keyboard device will (normally) return all of its queued bytes.
int write (fileDesc, buffer, size)	Analogous to read, write up to size bytes into kernel output buffer for a file, channel, or device. Write normally returns immediately but may stall if there is no space in the kernel buffer.
fileDesc select (fileDesc[], arraySize)	Return when any of the file descriptors in the array fileDesc[] have data available to be read. Returns the file descriptor that has data pending.
close (fileDescriptor)	Tell the kernel the process is done with this file, channel, or device.

Figure 3.7: List of UNIX system calls discussed in this section.

3.3 Case Study: Implementing a Shell

The dozen UNIX system calls listed in Figure 3.7 are enough to build a flexible and powerful command line shell, one that runs entirely at user-level with no special permissions. As we mentioned, the process that creates the shell is responsible for providing it an open file descriptor for reading commands for its input (e.g., from the keyboard), called *stdin* and for writing output (e.g., to the display), called *stdout*.

```
main() {
    char *prog = NULL;
    char **args = NULL;

    // Read the input a line at a time, and parse each line into the program
    // name and its arguments. End loop if we've reached the end of the input.
    while (readAndParseCmdLine(&prog, &args)) {

        // Create a child process to run the command.
        int child_pid = fork();

        if (child_pid == 0) {
            // I'm the child process.
            // Run program with the parent's input and output.
            exec(prog, args);
            // NOT REACHED
        } else {
            // I'm the parent; wait for the child to complete.
            wait(child_pid);
            return 0;
        }
    }
}
```

Figure 3.8: Example code for a simple UNIX shell.

Figure 3.8 illustrates the code for the basic operation of a shell. The shell reads a command line from the input, and it forks a process to execute that command. UNIX fork automatically duplicates all open file descriptors in the parent, incrementing the kernel's reference counts for those descriptors, so the input and output of the child is the same as the parent. The parent waits for the child to finish before it reads the next command to execute.

Because the commands to read and write to an open file descriptor are the same whether the file descriptor represents a keyboard, screen, file, device, or pipe, UNIX programs do not need to be aware of where their input is coming from, or where their output is going. This is helpful in a number of ways:

- **A program can be a file of commands.** Programs are normally a set of machine instructions, but on UNIX a program can be a file containing a list of commands for a shell to interpret. To disambiguate, shell programs signified in UNIX by putting "#! interpreter" as the first line of the file, where "interpreter" is the name of the shell executable.

  The UNIX C compiler works this way. When it is exec'ed, the kernel recognizes it as a shell file and starts the interpreter, passing it the file as input. The shell reads the file as a list of commands to invoke the pre-processor, parser, code generator and assembler in turn, exactly as if it was reading text input from the keyboard. When the last command completes, the shell interpreter calls exit to inform the kernel that the program is done.

- **A program can send its output to a file.** By changing the stdout file descriptor in the child, the shell can redirect the child's output to a file. In the standard UNIX shell, this is signified with a "greater than" symbol. Thus, "ls > tmp" lists the contents of the current directory into the file "tmp." After the fork and before the exec, the shell can replace the stdout file descriptor for the child using dup2. Because the parent has been cloned, changing stdout for the child has no effect on the parent.

- **A program can read its input from a file.** Likewise, by using dup2 to change the stdin file descriptor, the shell can cause the child to read its input from a file. In the standard UNIX shell, this is signified with a "less than" symbol. Thus, "zork < solution" plays the game "zork" with a list of instructions stored in the file "solution."

- **The output of one program can be the input to another program.** The shell can use a pipe to connect two programs together, so that the output of one is the input of another. This is called a *producer-consumer* relationship. For example, in the C-compiler, the output of the preprocessor is sent to the parser, and the output of the parser is sent to the code-generator and then to the assembler. In the standard UNIX shell, a pipe connecting two programs is signified by a "|" symbol, as in: "cpp file.c | cparse | cgen | as > file.o". In this case the shell creates four separate child processes, each connected by pipes to its predecessor and successor. Each of the phases can run in parallel, with the parent waiting for all of them to finish.

3.4 Case Study: Interprocess Communication

For many of the same reasons it makes sense to construct complex applications from simpler modules, it often makes sense to create applications that can specialize on a specific task, and then combine those applications into more complex structures. We gave an example above with the C compiler, but many parts of the operating system are structured this way. For example, instead of every program needing to know how to coordinate access to a printer, UNIX has a printer server, a specialized program for managing the printer queue.

For this to work, we need a way for processes to communicate with each other. Three widely used forms of interprocess communication are:

- **Producer-consumer.** In this model, programs are structured to accept as input the output of other programs. Communication is one-way: the producer only writes, and the consumer only reads. As we explained above, this allows chaining: a consumer can be, in turn, a producer for a different process. Much of the success of UNIX was due to its ability to easily compose many different programs together in this fashion.

- **Client-server.** An alternative model is to allow two-way communication between processes, as in client-server computing. The server implements some specialized task, such as managing the printer queue or managing the display. Clients send requests to the server to do some task, and when the operation is complete, the server replies back to the client.

- **File system.** Another way programs can be connected together is through reading and writing files. A text editor can import an image created by a drawing program, and the editor can in turn write an HTML file that a web server can read to know how to display a web page. A key distinction is that, unlike the first two modes, communication through the file system can be separated in *time*: the writer of the file does not need to be running at the same time as the file reader. Therefore, data needs to be stored persistently on disk or other stable storage, and the data needs to be named so that you can find the file when needed later on.

All three models are widely used both on a single system and over a network. For example, the Google MapReduce utility operates over a network in a producer-consumer fashion: the output of the map function is sent to the machines running the reduce function. The web is an example of client-server computing, and many enterprises and universities run centralized file servers to connect a text editor on one computer with a compiler running on another.

As persistent storage, file naming, and distributed computing are each complex topics in their own right, we defer the discussions of those topics to later chapters. Here we focus on interprocess communication, where both processes are running simultaneously on the same machine.

3.4.1 Producer-Consumer Communication

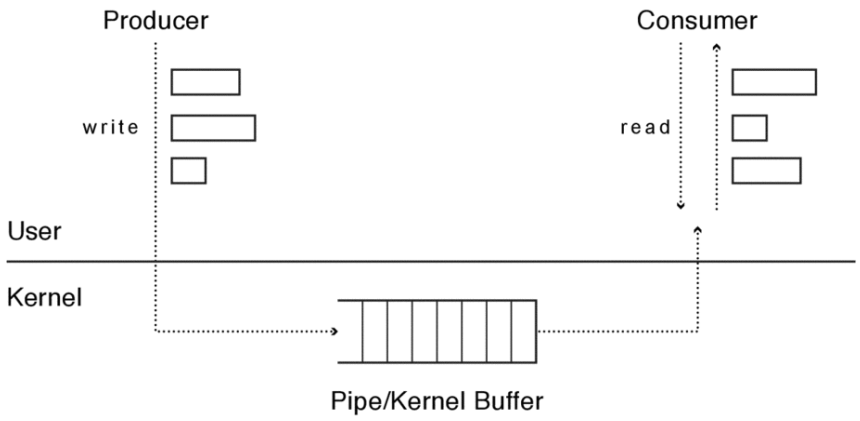


Figure 3.9: Interprocess communication between a producer application and a consumer. The producer uses the write system call to put data into the buffer; the consumer uses the read system call to take data out of the buffer.

Figure 3.9 illustrates how two processes communicate through the operating system in a producer-consumer relationship. Via the shell, we establish a pipe between the producer and the consumer. As one process computes and produces a stream of output data, it issues a sequence of write system calls on the pipe into the kernel. Each write can be of variable size. Assuming there is room in the kernel buffer, the kernel copies the data into the buffer, and returns immediately back to the producer.

At some point later, the operating system will schedule the consumer process to run. (On a multicore system, the producer and consumer could be running at the same time.) The consumer issues a sequence of read calls. Because the pipe is just a stream of bytes, the consumer can read the data out in any convenient chunking — the consumer can read in 1 KB chunks, while the producer wrote its data in 4 KB chunks, or vice versa. Each system call read made by the consumer returns the next successive chunk of data out of the kernel buffer. The consumer process can then compute on its input, sending its output to the display, a file, or onto the next consumer.

The kernel buffer allows each process to run at its own pace. There is no requirement that each process have equivalent amounts of work to do. If the producer is faster than the consumer, the kernel buffer fills up, and when the producer tries to write to a full buffer, the kernel stalls the process until there is room to store the data. Equivalently, if the consumer is faster than the producer, the buffer will empty and the next read request will stall until the producer creates more data.

In UNIX, when the producer finishes, it closes its side of the pipe, but there may still be data queued in the kernel for the consumer. Eventually, the consumer reads the last of the data, and the read system call will return an “end of file” marker. Thus, to the consumer, there is no difference between reading from a pipe and reading from a file.

Using kernel buffers to decouple the execution of the producer and consumer reduces the number and cost of context switches. Modern computers make extensive use of hardware caches to improve performance, but caches are ineffective if a program only runs for a short period of time before it must yield the processor to another task. The kernel buffer allows the operating system to run each process long enough to benefit from reuse, rather than alternating between the producer and consumer on each system call.

3.4.2 Client-Server Communication

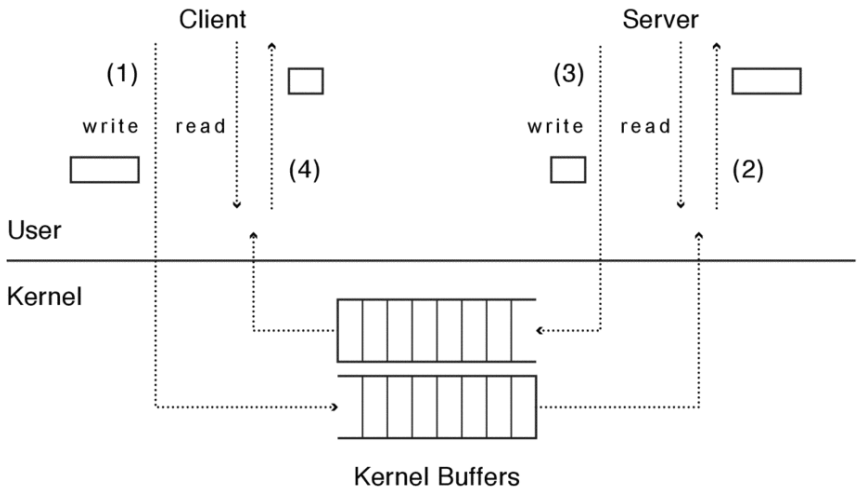


Figure 3.10: Interprocess communication between a client process and a server process. Once the client and server are connected, the client sends a request to the server by writing it into a kernel buffer. The server reads the request out of the buffer, and returns the result by writing it into a separate buffer read by the client.

We can generalize the above to illustrate client-server communication, shown in Figure 3.10. Instead of a single pipe, we create two, one for each direction. To make a request, the client writes the data into one pipe, and reads the response from the other. The server does the opposite: it reads requests from the first pipe, performs whatever is requested (provided the client has permission to make the request), and writes the response onto the second pipe.

The client and server code are shown in Figure 3.11. To simplify the code, we assume that the requests and responses are fixed-size.

```
Client:
    char request[RequestSize];
    char reply[ReplySize]

    // ..compute..

    // Put the request into the buffer.
```

```
    // Send the buffer to the server.
    write(output, request, RequestSize);

    // Wait for response.
    read(input, reply, ReplySize);

    // ..compute..

Server:
    char request[RequestSize];
    char reply[ReplySize];

    // Loop waiting for requests.
    while (1) {
        // Read incoming command.
        read(input, request, RequestSize);

        // Do operation.

        // Send result.
        write(output, reply, ReplySize);
    }
```

Figure 3.11: Example code for client-server interaction.

Streamlining client-server communication

Client-server communication is a common pattern in many systems, and so one can ask: how can we improve its performance? One step is to recognize that both the client and the server issue a write immediately followed by a read, to wait for the other side to reply; at the cost of adding a system call, these can be combined to eliminate two kernel crossings per round trip. Further, the client will always need to wait for the server, so it makes sense for it to donate its processor to run the server code, reducing delay. Microsoft added support for this optimization to Windows in the early 1990's when it converted to a microkernel design (explained a bit later in this chapter). However, as we noted earlier, modern computer architectures make extensive use of caches, so for this to work we need code and data for both the client and the server to be able to be in cache simultaneously. We will talk about mechanisms to accomplish that in a later chapter.

We can take this streamlining even further. On a multicore system, it is possible or even likely that both the client and server each have their own processor. If the kernel sets up a shared memory region accessible to both the client and the server and no other processes, then the client and server can (safely) pass requests and replies back and forth, as fast as the memory system will allow, without ever traversing into the kernel or relinquishing their processors.

Frequently, we want to allow many clients to talk to the same server. For example, there is one server to manage the print queue, although there can be many processes that want to be able to print. For this, the server uses the select system call to identify the pipe containing the request, as shown in Figure 3.12. The client code is unchanged.

```
Server:
    char request[RequestSize];
    char reply[ReplySize];
```

```
FileDescriptor clientInput[NumClients];
FileDescriptor clientOutput[NumClients];

// Loop waiting for a request from any client.
while (fd = select(clientInput, NumClients) {

    // Read incoming command from a specific client.
    read(clientInput[fd], request, RequestSize);

    // Do operation.

    // Send result.
    write(clientOutput[fd], reply, ReplySize);
}
```

Figure 3.12: Server code for communicating with multiple clients.

3.5 Operating System Structure

We started this chapter with a list of functionality that users and applications need from the operating system. We have shown that by careful design of the system call interface, we can offload some of the work of the operating system to user programs, such as to a shell or to a print server.

In the rest of this chapter, we ask how should we organize the remaining parts of the operating system. There are many dependencies among the modules inside the operating system, and there is often quite frequent interaction between these modules:

- Many parts of the operating system depend on synchronization primitives for coordinating access to shared data structures with the kernel.
- The virtual memory system depends on low-level hardware support for address translation, support that is specific to a particular processor architecture.
- Both the file system and the virtual memory system share a common pool of blocks of physical memory. They also both depend on the disk device driver.
- The file system can depend on the network protocol stack if the disk is physically located on a different machine.

This has led operating system designers to wrestle with a fundamental tradeoff: by centralizing functionality in the kernel, performance is improved and it makes it easier to arrange tight integration between kernel modules. However, the resulting systems are less flexible, less easy to change, and less adaptive to user or application needs. We discuss these tradeoffs by describing several options for the operating system architecture.

3.5.1 Monolithic Kernels

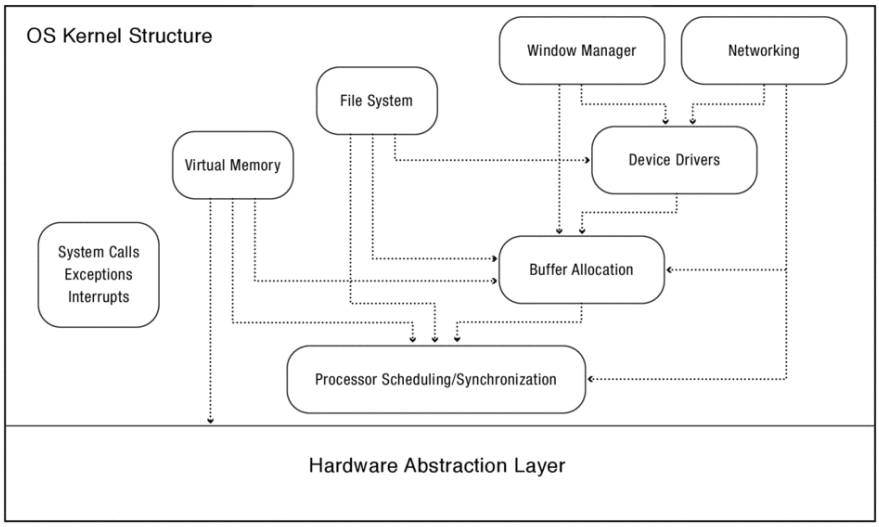


Figure 3.13: In a monolithic operating system kernel, most of the operating system functionality is linked together inside the kernel. Kernel modules directly call into other kernel modules to perform needed functions. For example, the virtual memory system uses buffer management, synchronization, and the hardware abstraction layer.

Almost all widely used commercial operating systems, such as Windows, MacOS, and Linux, take a similar approach to the architecture of the kernel — a monolithic design. As shown in Figure 3.13, with a *monolithic kernel*, most of the operating system functionality runs inside the operating system kernel. In truth, the term is a bit of a misnomer, because even in so-called monolithic systems, there are often large segments of what users consider the operating system that runs outside the kernel, either as utilities like the shell, or in system libraries, such as libraries to manage the user interface.

Internal to a monolithic kernel, the operating system designer is free to develop whatever interfaces between modules that make sense, and so there is quite a bit of variation from operating system to operating system in those internal structures. However, two common themes emerge across systems: to improve portability, almost all modern operating systems have both a hardware abstraction layer and dynamically loaded device drivers.

Hardware Abstraction Layer

A key goal of operating systems is to be portable across a wide variety of hardware platforms. To accomplish this, especially within a monolithic system, requires careful design of the *hardware abstraction layer*. The hardware abstraction layer (HAL) is a portable interface to machine configuration and processor-specific operations within the kernel. For example, within the same processor family, such as an Intel x86, different computer manufacturers will require different machine-specific code to configure and manage interrupts and hardware timers.

Operating systems that are portable across processor families, say between an ARM and an x86 or between a 32-bit and a 64-bit x86, will need processor-specific code for process

and thread context switches. The interrupt, processor exception, and system call trap handling is also processor-specific; all systems have those functions, but the specific implementation will vary. As we will see in Chapter 8, machines differ quite a bit in their architecture for managing virtual address spaces; most kernels provide portable abstractions on top of the machine-dependent routines, such as to translate virtual addresses to physical addresses or to copy memory from applications to kernel memory and vice versa.

With a well-defined hardware abstraction layer in place, most of the operating system is machine- and processor-independent. Thus, porting an operating system to a new computer is just a matter of creating new implementations of these low-level HAL routines and re-linking.

The hardware abstraction layer in Windows

As a concrete example, Windows has a two-pronged strategy for portability. To allow the same Windows kernel binary to be used across personal computers manufactured by different vendors, the kernel is dynamically linked at boot time with a set of library routines specifically written for each hardware configuration. This isolates the kernel from the specifics of the motherboard hardware.

Windows also runs across a number of different processor architectures. Typically, a different kernel binary is produced for each type of processor, with any needed processor-specific code; sometimes, conditional execution is used to allow a kernel binary to be shared across closely related processor designs.

Dynamically Installed Device Drivers

A similar consideration leads to operating systems that can easily accommodate a wide variety of physical I/O devices. Although there are only a handful of different instruction set architectures in wide use today, there are a huge number of different types of physical I/O devices, manufactured by a large number of companies. There is diversity in the hardware interfaces to devices as well as in the hardware chip sets for managing the devices. A recent survey found that approximately 70% of the code in the Linux kernel was in device-specific software.

To keep the rest of the operating system kernel portable, we want to decouple the operating system source code from the specifics of each device. For instance, suppose a manufacturer creates a new printer — what steps does the operating system manufacturer need to take to accommodate that change?

The key innovation, widely adopted today, is a *dynamically loadable device driver*. A dynamically loadable device driver is software to manage a specific device, interface, or chipset, added to the operating system kernel after the kernel starts running, to handle the devices that are present on a particular machine. The device manufacturer typically provides the driver code, using a standard interface supported by the kernel. The operating system kernel calls into the driver whenever it needs to read or write data to the device.

The operating system boots with a small number of device drivers — e.g., for the disk (to read the operating system binary into memory). For the devices physically attached to the

computer, the computer manufacturer bundles those drivers into a file it stores along with the bootloader. When the operating system starts up, it queries the I/O bus for which devices are attached to the computer and then loads those drivers from the file on disk. Finally, for any network-attached devices, such as a network printer, the operating system can load those drivers over the Internet.

While dynamically loadable device drivers solve one problem, they pose a different one. Errors in a device driver can corrupt the operating system kernel and application data structures; just as with a regular program, errors may not be caught immediately, so that user may be unaware that their data is being silently modified. Even worse, a malicious attacker can use device drivers to introduce a computer virus into the operating system kernel, and thereby silently gain control over the entire computer. Recent studies have found that 90% of all system crashes were due to bugs in device drivers, rather than in the operating system itself.

Operating system developers have taken five approaches to dealing with this issue:

- **Code inspection.** Operating system vendors typically require all device driver code to be submitted in advance for inspection and testing, before being allowed into the kernel.

- **Bug tracking.** After every system crash, the operating system can collect information about the system configuration and the current kernel stack, and sends this information back to a central database for analysis. Microsoft does this on a wide scale. With hundreds of millions of installed computers, even a low rate of failure can yield millions of bug reports per day. Many crashes happen inside the device driver itself, but even those that do not can sometimes be tracked down. For example, if failures are correlated with the presence of a particular device driver, or increase after the release of a new version of the driver, that can indicate the source of a problem.

- **User-level device drivers.** Both Apple and Microsoft strongly encourage new device drivers to run at user-level rather than in the kernel. Each device driver runs in a separate user-level process, using system calls to manipulate the physical device. This way, a buggy driver can only affect its own internal data structures and not the rest of the operating system kernel; if the device driver crashes, the kernel can restart it easily.

  Although user-level device drivers are becoming more common, it can be time-consuming to port existing device drivers to run at user-level. Unfortunately, there is a huge amount of existing device driver code that directly addresses internal kernel data structures; drawing a boundary around these drivers has proven difficult. Of course, supporting legacy drivers is less of a problem as completely new hardware and operating system platforms, such as smartphones and tablets, are developed.

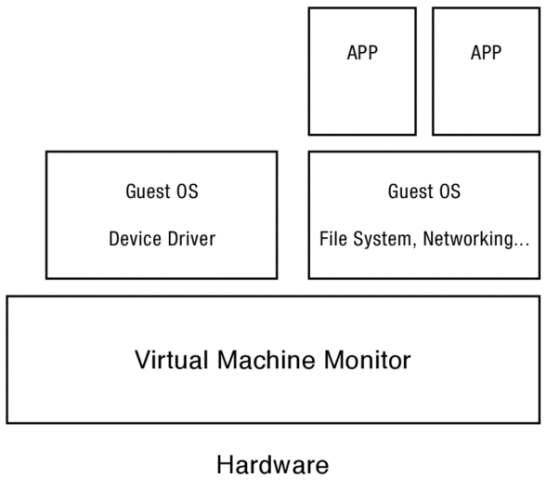


Figure 3.14: Legacy device drivers can run inside a guest operating system on top of a virtual machine in order to isolate the effect of implementation errors in driver code.

- **Virtual machine device drivers.** To handle legacy device drivers, one approach that has gained some traction is to run device driver code inside a guest operating system running on a virtual machine, as shown in Figure 3.14. The guest operating system loads the device drivers as if it was running directly on the real hardware, but when the devices attempt to access the physical hardware, the underlying virtual machine monitor regains control to ensure safety. Device drivers can still have bugs, but they can only corrupt the guest operating system and not other applications running on the underlying virtual machine monitor.

- **Driver sandboxing.** A further challenge for both user-level device drivers and virtual machine drivers is performance. Some device drivers need frequent interaction with hardware and the rest of the kernel. Some researchers have proposed running device drivers in their own restricted execution environment inside the kernel. This requires lightweight sandboxing techniques, a topic we will return to at the end of Chapter 8.

3.5.2 Microkernel

An alternative to the monolithic kernel approach is to run as much of the operating system as possible in one or more user-level servers. The window manager on most operating systems works this way: individual applications draw items on their portion of the screen by sending requests to the window manager. The window manager adjudicates which application window is in front or in back for each pixel on the screen, and then renders the result. If the system has a hardware graphics accelerator present, the window manager can use it to render items more quickly. Some systems have moved other parts of the operating system into user-level servers: the network stack, the file system, device drivers, and so forth.

The difference between a monolithic and a microkernel design is often transparent to the application programmer. The location of the service can be hidden in a user-level library — calls go to the library, which casts the requests either as system calls or as reads and writes to the server through a pipe. The location of the server can also be hidden inside the kernel — the application calls the kernel as if the kernel implements the service, but instead the kernel reformats the request into a pipe that the server can read.

A microkernel design offers considerable benefit to the operating system developer, as it easier to modularize and debug user-level services than kernel code. Aside from a potential reliability improvement, however, microkernels offer little in the way of visible benefit to end users and can slow down overall performance by inserting extra steps between the application and the services it needs. Thus, in practice, most systems adopt a hybrid model where some operating system services are run at user-level and some are in the kernel, depending on the specific tradeoff between code complexity and performance.

3.6 Summary and Future Directions

In this chapter, we have seen how system calls can be used by applications to create and manage processes, perform I/O, and communicate with other processes. Every operating system has its own unique system call interface; describing even a single interface in depth would be beyond the scope of this book. In this chapter, we focused parts of the UNIX interface because it is both compact and powerful. A key aspect of the UNIX interface are that creating a process (with fork) is separate from starting to run a program in that process (with exec); another key feature is the use of kernel buffers to decouple reading and writing data through the kernel.

Operating systems use the system call interface to provide services to applications and to aid in the internal structuring of the operating system itself. Almost all general-purpose computer systems today have a user-level shell and/or a window manager that can start and manage applications on behalf of the user. Many systems also implement parts of the operating system as user-level services accessed through kernel pipes.

As we noted, a trend is for applications to become mini-operating systems in their own right, with multiple users, resource sharing and allocation, untrusted third-party code, processor and memory management, and so forth. The system call interfaces for Windows and UNIX were not designed with this in mind, and an interesting question is how they will change to accommodate this future of powerful meta-applications.

In addition to the fine-grained sandboxing and process creation we described at the end of the last chapter, a trend is to re-structure the system call interface to make resource allocation decisions explicit and visible to applications. Traditionally, operating systems make resource allocation decisions — when to schedule a process or a thread, how much memory to give a particular application, where and when to store its disk blocks, when to send its network packets — transparently to the application, with a goal of improving end user and overall system performance. Applications are unaware of how many resources they have, appearing to run by themselves, isolated on their own (virtual) machine.

Of course, the reality is often quite different. An alternate model is for operating systems to divide resources among applications and then allow each application to decide for itself

how best to use those resources. One can think of this as a type of federalism. If both the operating system and applications are governments doing their own resource allocation, they are likely to get in each other's way if they are not careful. As a simple example, consider how a garbage collector works; it assumes it has a fixed amount of memory to manage. However, as other applications start or stop, it can gain or lose memory, and if the operating system does this reallocation transparently, the garbage collector has no hope of adapting. We will see examples of this same design pattern in many different areas of operating system design.

Exercises

1. Can UNIX fork return an error? Why or why not?
   Note: You can answer this question by looking at the manual page for fork, but before you do that, think about what the fork system call does. If you were designing this call, would you need to allow fork to return an error?

2. Can UNIX exec return an error? Why or why not?
   Note: You can answer this question by looking at the manual page for exec, but before you do that, think about what the exec system call does. If you were designing this call, would you need to allow it to return an error?

3. What happens if we run the following program on UNIX?

```
main() {
    while (fork() >= 0)
        ;
}
```

4. Explain what must happen for UNIX wait to return immediately (and successfully).

5. Suppose you were the instructor of a very large introductory programming class. Explain (in English) how you would use UNIX system calls to automate testing of submitted homework assignments.

6. What happens if you run "exec csh" in a UNIX shell? Why?

7. What happens if you run "exec ls" in a UNIX shell? Why?

8. How many processes are created if the following program is run?

```
main(int argc, char ** argv) {
    forkthem(5)
}
void forkthem(int n) {
    if (n > 0) {
        fork();
        forkthem(n-1);
    }
}
```

9. Consider the following program:

```
main (int argc, char ** argv) {
    int child = fork();
    int x = 5;

    if (child == 0) {
        x += 5;
    } else {
        child = fork();
        x += 10;
        if(child) {
            x += 5;
        }
    }
}
```

How many different copies of the variable x are there? What are their values when their process finishes?

10. What is the output of the following programs? (Please try to solve the problem without compiling and running the programs.)

```
// Program 1
main() {
    int val = 5;
    int pid;

    if (pid = fork())
        wait(pid);
    val++;
    printf("%d\n", val);
    return val;
}

// Program 2:
main() {
    int val = 5;
    int pid;
    if (pid = fork())
        wait(pid);
    else
        exit(val);
    val++;
    printf("%d\n", val);
    return val;
}
```

11. Implement a simple Linux shell in C capable of executing a sequence of programs that communicate through a pipe. For example, if the user types ls | wc, your program should fork off the two programs, which together will calculate the number of files in the directory. For this, you will need to use several of the Linux system calls described in this chapter: fork, exec, open, close, pipe, dup2, and wait. **Note:** You will to replace

stdin and stdout in the child process with the pipe file descriptors; that is the role of dup2.

12. Extend the shell implemented above to support foreground and background tasks, as well as job control: suspend, resume, and kill.

References

[1] Keith Adams and Ole Agesen. A comparison of software and hardware techniques for x86 virtualization. In Proceedings of the 12th International conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS-XII, pages 2–13, 2006.

[2] Thomas E. Anderson, Brian N. Bershad, Edward D. Lazowska, and Henry M. Levy. Scheduler activations: effective kernel support for the user-level management of parallelism. ACM Trans. Comput. Syst., 10(1):53–79, February 1992.

[3] Thomas E. Anderson, Henry M. Levy, Brian N. Bershad, and Edward D. Lazowska. The interaction of architecture and operating system design. In Proceedings of the fourth International conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS-IV, pages 108–120, 1991.

[4] Andrew W. Appel and Kai Li. Virtual memory primitives for user programs. In Proceedings of the fourth International conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS-IV, pages 96–107, 1991.

[5] Amittai Aviram, Shu-Chun Weng, Sen Hu, and Bryan Ford. Efficient system-enforced deterministic parallelism. In Proceedings of the 9th USENIX conference on Operating Systems Design and Implementation, OSDI'10, pages 1–16, 2010.

[6] Özalp Babaoglu and William Joy. Converting a swap-based system to do paging in an architecture lacking page-referenced bits. In Proceedings of the eighth ACM Symposium on Operating Systems Principles, SOSP '81, pages 78–86, 1981.

[7] David Bacon, Joshua Bloch, Jeff Bogda, Cliff Click, Paul Haahr, Doug Lea, Tom May, Jan-Willem Maessen, Jeremy Manson, John D. Mitchell, Kelvin Nilsen, Bill Pugh, and Emin Gun Sirer. The "double-checked locking is broken" declaration. http://www.cs.umd. edu/~pugh/java/memoryModel/DoubleCheckedLocking.html.

[8] Gaurav Banga, Peter Druschel, and Jeffrey C. Mogul. Resource containers: a new facility for resource management in server systems. In Proceedings of the third USENIX symposium on Operating Systems Design and Implementation, OSDI '99, pages 45–58, 1999.

[9] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. Xen and the art of virtualization. In Proceedings of the nineteenth ACM Symposium on Operating Systems Principles, SOSP '03, pages 164–177, 2003.

[10] Blaise Barney. POSIX threads programming. http://computing.llnl.gov/tutorials/pthreads/, 2013.

[11] Joel F. Bartlett. A nonstop kernel. In Proceedings of the eighth ACM Symposium on Operating Systems Principles, SOSP '81, pages 22–29, 1981.

[12] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhania. The multikernel: a new OS architecture for scalable multicore systems. In